

DRIFT TRIAGE CO-PILOT

Week 5 Project — Pairs (2 students)

A self-healing MLOps stack: model + agent + queue + dashboard

■ DEADLINE

Thursday midnight — submission

Friday — 5-minute presentation per pair

THE MISSION

Train a binary classifier on the **UCI Bank Marketing** dataset. Register it. Serve it. Watch it. Build a **LangGraph supervisor** that consumes the drift signal, decides what to do, dispatches slow consequences (replay the test set, retrain, rollback) through a Redis-backed queue, persists its state across restarts, and pauses for human approval before any change to Production reaches the registry.

This week ran two tracks: **MLOps** (artifacts, validation, observability with PSI / χ^2 , end-to-end serve and promotion) and **agentic systems** (multi-agent topologies, memory and persistence, agent SWE, long-running tools). Both are required. Neither is optional.

This is **not** a Colab notebook. It is a real repository that comes up with docker-compose up from a clean clone. The platform and the agent each survive the other crashing. The agent's decision logic is regression-tested in CI. The queue is idempotent. None of these are stretch goals.

You and your partner divide labour however you want. Both names go on the repo, both names answer questions on Friday.

ARCHITECTURE

FastAPI + MLflow + drift | LangGraph supervisor + Postgres checkpoints | Redis

queue + DLQ — one Streamlit dashboard

DATASET

UCI Bank Marketing — phone-call campaign records from a Portuguese retail bank. ~41,188 rows × 20 features. Target: did the client subscribe to a term deposit? (~11% positive). Use bank-additional-full.csv. Stratified 60/20/20 split, random_state=42.

Known traps: drop duration — it's recorded after the call ends and leaks the target. Treat pdays==999 as a sentinel (bin or flag). Treat 'unknown' as a real category, not as missing data —

it is informative.

Drift narrative for Friday: the macroeconomic features (euribor3m, cons.price.idx) are real economic indicators. Shift one numeric and one categorical live during the demo and watch both the platform and the agent react.

WHAT THE PLATFORM MUST DO

- Train a pipeline (preprocessor + classifier) and tune an operating threshold by the rule from week 5 day 2 (highest threshold meeting recall ≥ 0.75).
- Register the fitted pipeline in MLflow with the standard artifact triple — binary, schema, and a model card with hash + environment fingerprint.
- Serve predictions over HTTP with Pydantic-validated requests. Bad inputs return structured errors, never stack traces.
- Compute a drift report over a rolling window of recent predictions: PSI on numerics, χ^2 on categoricals, output-distribution drift.
- Emit a webhook to the agent when the drift report changes severity.
- Refuse to promote any version to Production except through a programmatic gate that asserts the day-4 promotion checklist.
- Persist registry state, predictions, and reference statistics across container restarts.

WHAT THE AGENT MUST DO

- Subscribe to drift webhooks (or poll — defend the choice). Each new event becomes a new investigation.
- Use a true **supervisor** topology — three sub-agents (triage, action, comms), not a chain.
- Persist state in Postgres via LangGraph checkpoints. Killing the agent mid-investigation and restarting it must resume from the last checkpoint, not start over.
- **Pause for human approval** before any action that touches Production. The HIL surface lives in the dashboard.
- Dispatch slow tools (replay test, retrain, rollback) through a Redis queue with idempotency keys, exponential-backoff retries, and a dead-letter queue.
- Have routing logic and full trajectories under regression test in CI. Snapshot trajectories must match recorded fixtures; mock the LLM so the test runs without an API key.
- Treat prompts as code: stored as files in version control, never as inline strings.

INTEGRATION

The platform and the agent agree on a contract: an HTTP webhook from the platform to the agent on every alert, and an HTTP call from the agent back to the platform's promotion endpoint when an action is approved by a human. The contract is yours to design — write it down, version it, and treat schema changes as breaking.

The dashboard surfaces both halves: registry state, agent's open and resolved investigations, the queue depth and DLQ, and the HIL inbox where pending approvals appear.

The full stack — model service, agent, queue worker, dashboard, Postgres, Redis — comes up with docker-compose up from a clean clone. CI runs on every push: build images, run agent

snapshot trajectory tests with a mocked LLM, run a 1e-12 fidelity replay test against the model. Refuse to merge if any of these regress.

FRIDAY PRESENTATION

5 minutes. Architecture walkthrough → live demo (drift the input, watch the agent open an investigation, hit a HIL approval, watch the queue dispatch the action, watch the dashboard reflect the outcome) → show one CI failure on a snapshot trajectory regression → one specific real bug you hit and fixed.

We will ask **both of you** to explain decisions on the demo. We will ask **one of you** to explain the other's code.

THINK ABOUT

How do you keep the agent's checkpoint store and the model registry in sync when one rolls back and the other doesn't? What does the agent do when it wakes from a checkpoint and the model URI it was investigating no longer exists? How do you guarantee that two retries of the same retrain don't kick off two trainings? When the dashboard shows a HIL approval for "promote v2 to Production," what stops the agent from acting on a stale recommendation if a newer drift event has already arrived? Can the platform's promotion endpoint be called without going through the agent — and should it?

These are your problems to solve. No hints.

SUBMISSION

Public GitHub repository. Tag a release v0.1.0-week5 before Thursday midnight. The repo must come up cleanly with docker-compose up from a fresh clone after a cp .env.example .env and filling in the required secrets.

Format your submission message as:

Project 5 - [Name 1] | [Name 2]

Repo: [GitHub URL]

Tag: v0.1.0-week5

Dataset: UCI Bank Marketing (bank-additional-full.csv)

Model: [registered name + version] (Test AUC: [n] | Test F1: [n]) Operating threshold: [n]

(rule: recall $\geq$ 0.75)

LLM: [provider + model] - chosen because [one line]

README contains: ARCH.md, DECISIONS.md, RUNBOOK.md

RULES

■ NO VIBE CODING

Understand every line you ship. Both of you will be asked about it on Friday.

Ship it.

Thursday midnight. No excuses.